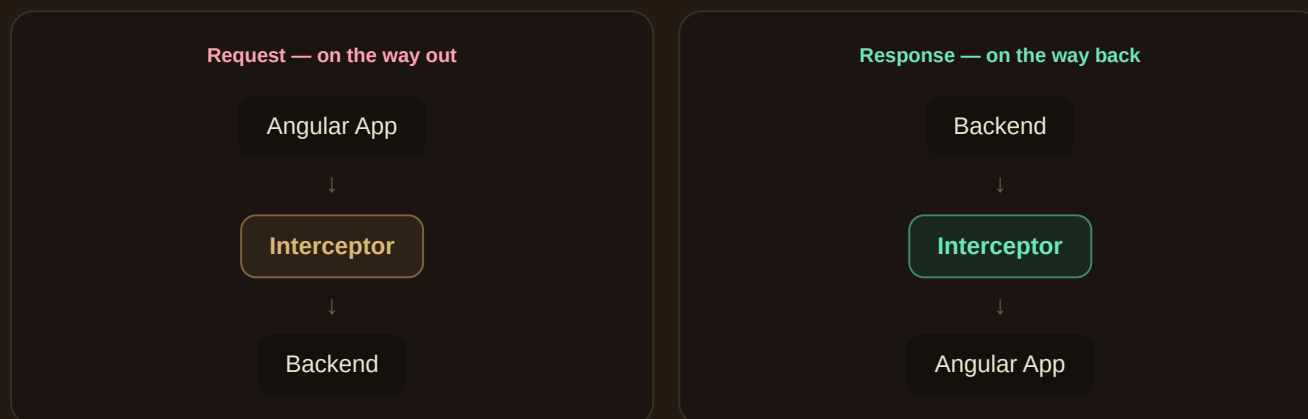


HttpInterceptor

An `HttpInterceptor` is **one function standing in the middle** — every request your app sends passes through it on the way out, and every response passes through it on the way back, before your component ever sees it.

1 · WHAT IS AN INTERCEPTOR?

One function standing in the middle



The request passes through the interceptor before it ever reaches the backend — and the response (or the error) passes through it again on the way back.

2 · A SIMPLE ANALOGY

A security guard standing at the door

Imagine you're walking into a company building. Before you reach your desk, there's a **Security Guard** at the door.



The guard might say:

- "Show me your ID."
- "Wear your badge."
- "You can't come in."

The interceptor is exactly that guard — it inspects every HTTP request before it's allowed to reach the backend.

3 · THE MOST COMMON USE — ATTACHING A TOKEN

Every request needs an Authorization header

Without an interceptor, every service method that calls the API would need to remember to read the token and attach it manually. With one, it happens automatically, in a single place:

① Original request — built by the component

GET /api/users
Host: api.example.com

No `Authorization` header — the component never thinks about the token.

② Cloned request — after the interceptor

GET /api/users
Host: api.example.com
Authorization: Bearer eyJhbGciOi...

`req.clone({ setHeaders: ... })` — a new request object, token attached.

4 · THE CODE

authInterceptor — line by line

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  
  const token = localStorage.getItem('token');  
  
  const clonedRequest = req.clone({  
    setHeaders: { Authorization: `Bearer ${token}` }  
  });  
  
  return next(clonedRequest);  
};
```

REQ

The current HTTP request — e.g. GET /api/users. It's read-only.

NEXT

Means "continue the request through the pipeline" — call it once you're done inspecting or modifying.

5 · WHY CLONE()?

HttpRequest objects are immutable

A common question: "why not just edit `req` directly?" — because in Angular, an `HttpRequest` can't be changed in place. Instead, you create a modified *copy*:

Original request

→

`req.clone({ ... })`

→

Modified request

✅ return next(clonedRequest)

"I'm done editing — send the modified version onward."

❌ return next(req)

Sends the original, unmodified request — your token never gets attached.

Not just requests — the response flows through it as well

Imagine the backend replies with **401 Unauthorized** — usually meaning the token is invalid or expired. The interceptor sees that status on the way back and reacts, before your component even gets the error:

```
return next(clonedRequest).pipe(
  catchError((error: HttpResponse) => {
    if (error.status === 401) {
      localStorage.removeItem('token');
      router.navigate(['/login']);
    }
    return throwError(() => error);
  })
);
```

Depending on the app's architecture, that reaction could be a token refresh, a forced logout, a redirect to `/login`, or a shared error toast — decided in **one place**, instead of every component that calls the API.

7 · NOT JUST FOR AUTHENTICATION

The same "one checkpoint" idea, reused for other things

USE CASE	WHAT IT DOES
Authentication	Attach the Authorization header to every request automatically.
Logging	Record every request URL and its response status for debugging.
Error handling	Catch 401 / 403 / 500 in one place instead of every component.
Loading indicator	Show a spinner when a request starts, hide it when it finishes.
Common headers	Add Content-Type, Accept-Language, or a Correlation-ID to every call.